

Assessment -2

Section A — Concept Application

S1. Statelessness in REST

In REST, statelessness means that every client request must contain all the information required to process it. The server does not store any client session information between requests. Each request is treated as completely independent.

For a multi-city food delivery API, this means the server does not remember whether a user logged in previously. Instead, the client (mobile app or web frontend) sends an authentication token (such as a JWT token) with every request. The server validates the token and processes the request without relying on stored session data.

This approach improves scalability because any server instance can handle any request without sharing session information. It also makes load balancing easier and reduces server memory usage.

Real-world example: A customer logs into the food delivery app and receives a JWT token. When requesting `/orders/`, the mobile app includes the token in the Authorization header. Even if the request is handled by a different server, the API can authenticate the user because all required information is included in that request.

S2. ModelSerializer vs Serializer

For a restaurant management endpoint where new menu items are created, ModelSerializer is the better choice because it automatically generates serializer fields based on the Django model and includes built-in create and update methods. This reduces boilerplate code, improves maintainability, and ensures consistency with the database model.

To protect menu item data integrity, I would add these field-level validations:

➤ Price Validation

- Ensure the price is greater than zero.
- Prevent negative or zero-priced menu items.

➤ Name Validation

- Ensure the dish name is not empty and has a minimum length (for example, at least 3 characters).
- Prevent duplicate or invalid menu item names.

These validations help maintain clean, accurate, and reliable data in the restaurant database.

S3. **ModelViewSet and DefaultRouter**

Using a `ModelViewSet` with a `DefaultRouter` significantly reduces URL management because a single `ViewSet` automatically provides all CRUD operations instead of creating separate `APIView` classes and URL patterns for each action.

For example, instead of writing separate URLs for list, create, retrieve, update, and delete operations, the router generates them automatically.

A `ModelViewSet` automatically handles these HTTP methods:

- **GET** – Retrieve one or multiple resources
- **POST** – Create a new resource
- **PUT** – Update an entire resource
- **PATCH** – Partially update a resource
- **DELETE** – Remove a resource

This approach results in cleaner code, less duplication, and easier maintenance as the API grows.

S4. Pagination Performance

The most likely reason for the slow response is that the API is returning all available dishes from over 500 restaurants in a single response. Fetching and serializing thousands of records increases database queries, processing time, and network transfer size.

Comparison

➤ **PageNumberPagination**

- Simple to implement and understand.
- Allows users to jump directly to any page.
- Suitable for most CRUD applications.
- Can become slower with very large datasets because the database must count records.

➤ **CursorPagination**

- More efficient for very large datasets.
- Provides consistent pagination even when new records are added.
- Does not allow users to jump directly to a specific page.
- Ideal for APIs with continuously changing data.

For a dish search endpoint with a very large and frequently updated dataset, CursorPagination is generally the better long-term choice because it offers better performance and consistency.

S5. Object-Level Permission

The gap is that `IsAuthenticated` only checks whether the user is logged in. It does not verify whether the authenticated user owns the requested order.

To solve this problem, I would implement a custom object-level permission by creating a permission class that checks whether the order belongs to the current user.

Example logic:

```
from rest_framework.permissions import BasePermission

class IsOrderOwner(BasePermission):

    def has_object_permission(self, request, view, obj):

        return obj.customer == request.user
```

Then apply this permission to the order view.

When a customer requests an order, DRF first authenticates the user and then calls `has_object_permission()`. If the logged-in user is not the owner of the order, the API returns HTTP 403 Forbidden.

This ensures that customers can only view or modify their own orders.

S6. Google Maps Geocoding API Integration

To implement a "Find Restaurants Near Me" feature, I would integrate the Google Maps Geocoding API into a DRF view.

The process would be:

- The user submits an address (for example, "IIM Ahmedabad, Gujarat").
- The DRF view sends the address and API key to the Google Geocoding API.
- Google returns the latitude and longitude of the address.
- The API calculates the distance between the user's coordinates and each restaurant's stored coordinates using the Haversine formula or a geospatial query.
- Restaurants within a specified radius (such as 5 km) are returned, sorted by distance.